Python Basics · Day 3 of 30 · about 60 minutes

Lists and dictionaries

🎯 Today's goal

By the end of this hour you will:

- keep many values in one variable with a **list**
- pick out items by their position, and cut out pieces of a list with **slicing**
- add, insert and remove items, and use `sum`, `min`, `max` and `sorted`
- store labelled values in a **dictionary**, like a price list or a student record
- combine the two into a tiny table: a list of dictionaries

🤔 Why this matters

Yesterday one variable held one value. But data almost never comes one value at a time: a week has seven temperatures, a class has thirty students, a shop has hundreds of prices.

A **list** is like a numbered row of boxes: the values stay in order and you find each one by its position. A **dictionary** is like a real dictionary or a phone book: you look things up by a *word* (the key) instead of a number. A spreadsheet column is basically a list, and one row of a spreadsheet (column name → value) is basically a dictionary. Later in the course, pandas tables are built from exactly these two ideas.

📋 Your first list

A list is written with **square brackets** `[ ]`, with commas between the items. Here is a (made-up) week of midday temperatures, Monday to Sunday:

```
temps = [21.5, 23.0, 19.8, 25.1, 27.3, 26.0, 22.4]

print(temps)
print(len(temps))      # len() counts the items in a list
print(type(temps))
```

```
[21.5, 23.0, 19.8, 25.1, 27.3, 26.0, 22.4]
7
<class 'list'>
```

Each value in a list is called an **item** (or element). A list can hold any type: numbers, text, booleans, even other lists. A list of text looks like this: `days = ["Mon", "Tue", "Wed"]`.

🔢 Getting one item: indexing

Every item has a position number called its **index**. Python starts counting at **0**, not 1. So the first item is `temps[0]`, the second is `temps[1]`, and so on.

Day	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Index	0	1	2	3	4	5	6
From the end	-7	-6	-5	-4	-3	-2	-1

```
print("Monday:", temps[0])
print("Wednesday:", temps[2])

# Negative numbers count from the end: -1 is the last item
print("Sunday:", temps[-1])
print("Saturday:", temps[-2])
```

```
Monday: 21.5
Wednesday: 19.8
Sunday: 22.4
Saturday: 26.0
```

A list has 7 items, so the indexes go from 0 to 6. Asking for index 7 is asking for an eighth day that doesn't exist. Python answers with an `IndexError`:

💡 This example has a mistake on purpose. Run it and read the error.

```
temps[7]
```

⚠️ IndexError

You asked for a position that doesn't exist. Counting starts at 0.

```
Traceback (most recent call last):
  line 1, in <module>
    temps[7]
    ~~~~~^^^
IndexError: list index out of range
```

Read the last line of the error: *list index out of range*. That means "there is no item at that position". It's almost always an off-by-one mistake: remember that the last index is `len(temps) - 1`, or just

use `-1`.

✂️ Getting several items: slicing

A **slice** cuts out a piece of a list. You write `temps[start:stop]`. It starts *at* `start` and stops *just before* `stop`, so `temps[0:3]` gives the items at index 0, 1 and 2. Leave out a number to mean "from the very start" or "to the very end":

```
print(temps[0:3])     # Mon, Tue, Wed (index 0, 1, 2; not 3)
print(temps[:3])      # the same: an empty start means "from the start"
print(temps[5:])      # from index 5 to the end: Sat, Sun
print(temps[-3:])     # the last three days
print(temps[1:6])     # Tue to Sat

print(temps)          # slicing makes a new list; the original is unchanged
```

```
[21.5, 23.0, 19.8]
[21.5, 23.0, 19.8]
[26.0, 22.4]
[27.3, 26.0, 22.4]
[23.0, 19.8, 25.1, 27.3, 26.0]
[21.5, 23.0, 19.8, 25.1, 27.3, 26.0, 22.4]
```

Tip: `stop - start` is how many items you get. `temps[1:6]` gives 6 – 1 = 5 items.

✏️ Changing a list

Lists can grow and shrink. These list **methods** (tools you call with a dot, like yesterday's `.strip()`) change the list itself:

Method	What it does
`queue.append(x)`	adds `x` at the end
`queue.insert(i, x)`	puts `x` at index `i`, pushing the rest along
`queue.remove(x)`	removes the first item equal to `x`
`queue.pop()`	takes out the last item and gives it back to you
`queue.pop(i)`	takes out the item at index `i` and gives it back

Here is a (made-up) waiting list at a clinic:

```
queue = ["Amir", "Bea", "Chen"]

queue.append("Dina")         # a new patient arrives
print(queue)

queue.insert(0, "Emil")      # an emergency goes to the front
print(queue)

queue.remove("Bea")          # Bea went home
print(queue)

next_patient = queue.pop(0)  # take the first person out of the list
print("Now seeing:", next_patient)
print("Still waiting:", queue)
```

```
['Amir', 'Bea', 'Chen', 'Dina']
['Emil', 'Amir', 'Bea', 'Chen', 'Dina']
['Emil', 'Amir', 'Chen', 'Dina']
Now seeing: Emil
Still waiting: ['Amir', 'Chen', 'Dina']
```

You can also replace one item by giving its index a new value:

```
queue[0] = "Farah"        # replace whoever is at index 0
print(queue)

# "in" asks: is this value somewhere in the list? The answer is True or False.
print("Chen" in queue)
print("Bea" in queue)
```

```
['Farah', 'Chen', 'Dina']
True
False
```

🧮 Quick statistics on a list

Python has built-in functions that work on a whole list of numbers at once. This is your first taste of real data analysis:

```
print("Total:", sum(temps))
print("Coldest:", min(temps))
print("Hottest:", max(temps))

average = sum(temps) / len(temps)
print(f"Average: {average:.1f} degrees")
```

```
Total: 165.1
Coldest: 19.8
Hottest: 27.3
Average: 23.6 degrees
```

To put a list in order there are two ways. `sorted(temps)` gives you a **new** sorted list and leaves the original alone. `temps.sort()` sorts the list **itself** and gives back nothing. Add `reverse=True` for biggest first. Text is sorted alphabetically.

```
print(sorted(temps))                  # a new list, smallest first
print(sorted(temps, reverse=True))    # biggest first
print(temps)                          # unchanged: still Monday to Sunday

cities = ["Lima", "Cairo", "Oslo", "Accra"]
cities.sort()                         # changes the list itself
print(cities)
```

```
[19.8, 21.5, 22.4, 23.0, 25.1, 26.0, 27.3]
[27.3, 26.0, 25.1, 23.0, 22.4, 21.5, 19.8]
[21.5, 23.0, 19.8, 25.1, 27.3, 26.0, 22.4]
['Accra', 'Cairo', 'Lima', 'Oslo']
```

Careful: `temps = temps.sort()` is a classic mistake. `.sort()` gives back nothing (Python's word for "nothing" is `None`), so your list would be replaced by `None`. Use either `temps.sort()` on its own line, or `new_list = sorted(temps)`.

📖 Dictionaries: looking things up by name

A **dictionary** (`dict` for short) stores pairs of a **key** and a **value**. You write it with curly brackets `{}`, a colon between each key and its value, and commas between the pairs. Here is a (made-up) price list from a fruit stall, in dollars per kilo:

```
prices = {"apple": 2.5, "banana": 1.2, "cherry": 8.0}

print(prices)
print(prices["banana"])     # look up a value by its key, with square brackets
print(len(prices))          # how many key-value pairs
```

```
{'apple': 2.5, 'banana': 1.2, 'cherry': 8.0}
1.2
3
```

To **add** a new pair, or **update** an existing one, you use the same line: `prices[key] = value`. If the key is new it's added; if it already exists its value is replaced. `del` deletes a pair:

```
prices["kiwi"] = 3.1        # new key: added
prices["apple"] = 2.8       # existing key: updated
prices["banana"] += 0.3     # the shortcuts from Day 2 work too
del prices["cherry"]        # removed

prices
```

```
{'apple': 2.8, 'banana': 1.5, 'kiwi': 3.1}
```

Asking for a key that isn't in the dictionary gives a `KeyError`:

💡 This example has a mistake on purpose. Run it and read the error.

```
prices["mango"]
```

⚠️ KeyError

That key or column name doesn't exist. Check the spelling.

```
Traceback (most recent call last):
  line 1, in <module>
    prices["mango"]
    ~~~~~~^^^^^^^^^
KeyError: 'mango'
```

The error shows the missing key: `'mango'`. There are two polite ways to avoid it. `in` checks whether a key exists, and `.get(key, default)` gives back the value if the key exists, or the **default** (a back-up value you choose) if it doesn't:

```
print("mango" in prices)          # False: no such key
print("kiwi" in prices)           # True

print(prices.get("kiwi", 0))      # the key exists: 3.1
print(prices.get("mango", 0))     # missing: the default, 0
print(prices.get("mango"))        # no default given: None
```

```
False
True
3.1
0
None
```

You can ask a dictionary for all its keys, all its values, or all its pairs (called **items**). Each pair is shown in round brackets, like `('apple', 2.8)`:

```
print(prices.keys())
print(prices.values())
print(prices.items())

print("Most expensive price:", max(prices.values()))
print("Fruits we sell:", sorted(prices.keys()))
```

```
dict_keys(['apple', 'banana', 'kiwi'])
dict_values([2.8, 1.5, 3.1])
dict_items([('apple', 2.8), ('banana', 1.5), ('kiwi', 3.1)])
Most expensive price: 3.1
Fruits we sell: ['apple', 'banana', 'kiwi']
```

🗂️ Putting them together

A dictionary's values can be anything, including a list. That's perfect for a **record**: everything we know about one (made-up) student, with all their test scores:

```
student = {
    "name": "Lina",
    "age": 21,
    "scores": [85, 92, 78],
}

print(student["name"])
print(student["scores"])         # the whole list
print(student["scores"][1])      # then index into it: the second score

student["scores"].append(88)     # a new test result
mean_score = sum(student["scores"]) / len(student["scores"])
print(f"{student['name']} has an average of {mean_score:.1f}")
```

```
Lina
[85, 92, 78]
92
Lina has an average of 85.8
```

Quotes inside f-strings: the f-string above uses double quotes outside, so the key inside the curly brackets uses single quotes: `{student['name']}`. Python accepts both kinds of quotes; they just must not clash.

And a **list of dictionaries** is a tiny table: each dictionary is one row, and the keys are the column names. Here are three made-up patients:

```
patients = [
    {"name": "Amir", "age": 34, "temp": 37.1},
    {"name": "Bea",  "age": 58, "temp": 38.4},
    {"name": "Chen", "age": 25, "temp": 36.8},
]

print(len(patients), "patients")
print(patients[1])               # the second row
print(patients[1]["temp"])       # one cell: row 1, column "temp"

patients.append({"name": "Dina", "age": 41, "temp": 37.6})
print(patients[-1]["name"], "was added")
```

```
3 patients
{'name': 'Bea', 'age': 58, 'temp': 38.4}
38.4
Dina was added
```

Read `patients[1]["temp"]` from left to right: "from the patients list take item 1, then from that dictionary take the value of `temp`." This is the shape a lot of real data has when it arrives from the

internet (a format called JSON).

⚠️ Common mistakes

- Counting from 1. The first item is `[0]` and the last is `[-1]`.
- Forgetting that a slice stops *before* the stop number: `temps[0:3]` is 3 items, not 4.
- Writing `temps = temps.sort()`. The list becomes `None`. Use `temps.sort()` alone, or `sorted(temps)`.
- Mixing up brackets: lists use `[ ]`, dictionaries use `{ }`, but you *look things up* in both with `[ ]`.
- Misspelling a key, or using a different capital letter (`"Apple"` is not `"apple"`) → `KeyError`. Use `.get()` when a key might be missing.

✅ Recap

- A list keeps items in order: `temps = [21.5, 23.0, 19.8]`.
- Indexing starts at 0; `-1` is the last item. A slice `[start:stop]` stops just before `stop`.
- `.append()`, `.insert()`, `.remove()`, `.pop()` change a list; `in` checks membership.
- `len`, `sum`, `min`, `max`, `sorted` work on whole lists.
- A dictionary maps keys to values: `prices = {"apple": 2.5}`. Look up with `prices["apple"]`, add or update with `prices["kiwi"] = 3.1`, delete with `del`.
- `.get(key, default)` avoids a `KeyError`; `.keys()`, `.values()`, `.items()` show what's inside.
- A list of dictionaries is a small table: `patients[1]["temp"]`.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Finish the week

The list `temps` has the temperatures from Monday to Saturday. Sunday was `24.0` degrees. Add it to the **end** of the list with `.append()`, so the list has 7 items.

```
temps = [18.5, 20.1, 22.3, 21.0, 19.4, 23.8]

# Add Sunday's temperature
```

Hints:

1. A method is used with a dot after the list's name: `temps.append(...)`
2. Put the value inside the brackets: `temps.append(24.0)`

2. Weekdays and weekend

Using **slicing**, store the first five days (Monday to Friday) in a variable called `weekdays`, and the last two days (Saturday and Sunday) in a variable called `weekend`. Don't type the numbers again: cut them out of `temps`.

```
temps = [18.5, 20.1, 22.3, 21.0, 19.4, 23.8, 24.0]

# Slice out weekdays and weekend
```

Hints:

1. A slice looks like `temps[start:stop]` and stops just *before* `stop`.
2. The first five: `temps[:5]`. The last two: `temps[-2:]` (or `temps[5:]`).
3. `weekdays = temps[:5]` and `weekend = temps[5:]`

3. The week in numbers

Using the list functions from today, create three variables:

- `hottest` : the highest temperature
- `coldest` : the lowest temperature
- `average` : the average temperature, **rounded to 1 decimal** with `round()`

Your code should still work if someone changes the numbers in the list.

```
temps = [18.5, 20.1, 22.3, 21.0, 19.4, 23.8, 24.0]

# Calculate hottest, coldest and average
```

Hints:

1. `max(temps)` and `min(temps)` give the highest and lowest values.
2. The average is the total divided by how many: `sum(temps) / len(temps)`
3. `average = round(sum(temps) / len(temps), 1)`

4. A student record

Work with a dictionary called `student`, in three steps:

1. Create it with the keys `"name"` → `"Omar"`, `"age"` → `20` and `"course"` → `"Data Science"`.
2. It's Omar's birthday: on a new line, **update** his age to `21`.
3. Add a new key `"city"` with the value `"Tunis"`.

```
# 1. Create the dictionary


# 2. Update the age


# 3. Add the city
```

Hints:

1. A dictionary: `student = {"name": "Omar", "age": 20, ...}` with commas between the pairs.
2. Update a value with `student["age"] = 21` (or `student["age"] += 1`).
3. Adding a key uses the same pattern: `student["city"] = "Tunis"`

5. Price lookup

A customer wants bananas and mangoes. Use `.get()` with a default of `0` to store the price of bananas in `banana_price` and the price of mangoes in `mango_price`. (The stall doesn't sell mangoes, so that one should be 0 instead of a `KeyError`.)

```
prices = {"apple": 2.5, "banana": 1.2, "cherry": 8.0, "kiwi": 3.1}

# Look up both prices with .get()
```

Hints:

1. `.get()` takes the key first and the default second: `prices.get(key, default)`
2. `banana_price = prices.get("banana", 0)`
3. Do the same for `"mango"`.

6. Challenge: patient BMI

BMI (body mass index) is weight in kilos divided by height in metres squared: `weight / height ** 2`. Using the values **from the dictionary**:

1. calculate the BMI in a variable called `bmi`
2. store it in the dictionary too, under the key `"bmi"`, rounded to 1 decimal
3. print a report like `Hana: BMI 22.9` using an f-string

Your code should still work if someone changes the patient's name, weight or height. (Made-up patient. BMI is a teaching example here, not medical advice.)

```
patient = {"name": "Hana", "weight_kg": 70, "height_m": 1.75}

# Calculate bmi, store it in the dictionary, print the report
```

Hints:

1. Get the values with `patient["weight_kg"]` and `patient["height_m"]`.
2. `bmi = patient["weight_kg"] / patient["height_m"] ** 2`, then `patient["bmi"] = round(bmi, 1)`
3. Inside a double-quoted f-string, use single quotes for the key: `print(f"{patient['name']}: BMI {bmi:.1f}")`

➡️ Tomorrow

Making decisions with if / else: so far Python runs every line, every time. Tomorrow your code learns to choose: "if the temperature is over 38, print a fever warning, otherwise say all is well." You'll use the `True` / `False` values from Day 2 and the `in` checks from today to make decisions.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Finish the week

```
temps = [18.5, 20.1, 22.3, 21.0, 19.4, 23.8]

# Add Sunday's temperature
temps.append(24.0)
```

2. Weekdays and weekend

```
temps = [18.5, 20.1, 22.3, 21.0, 19.4, 23.8, 24.0]

# Slice out weekdays and weekend
weekdays = temps[:5]
weekend = temps[5:]
```

3. The week in numbers

```
temps = [18.5, 20.1, 22.3, 21.0, 19.4, 23.8, 24.0]

hottest = max(temps)
coldest = min(temps)
average = round(sum(temps) / len(temps), 1)
```

4. A student record

```
# 1. Create the dictionary
student = {"name": "Omar", "age": 20, "course": "Data Science"}

# 2. Update the age
student["age"] = 21

# 3. Add the city
student["city"] = "Tunis"
```

5. Price lookup

```
prices = {"apple": 2.5, "banana": 1.2, "cherry": 8.0, "kiwi": 3.1}

banana_price = prices.get("banana", 0)
mango_price = prices.get("mango", 0)
```

6. Challenge: patient BMI

```
patient = {"name": "Hana", "weight_kg": 70, "height_m": 1.75}

bmi = patient["weight_kg"] / patient["height_m"] ** 2
patient["bmi"] = round(bmi, 1)
print(f"{patient['name']}: BMI {bmi:.1f}")
```